UNIT-II
Process Synchronization

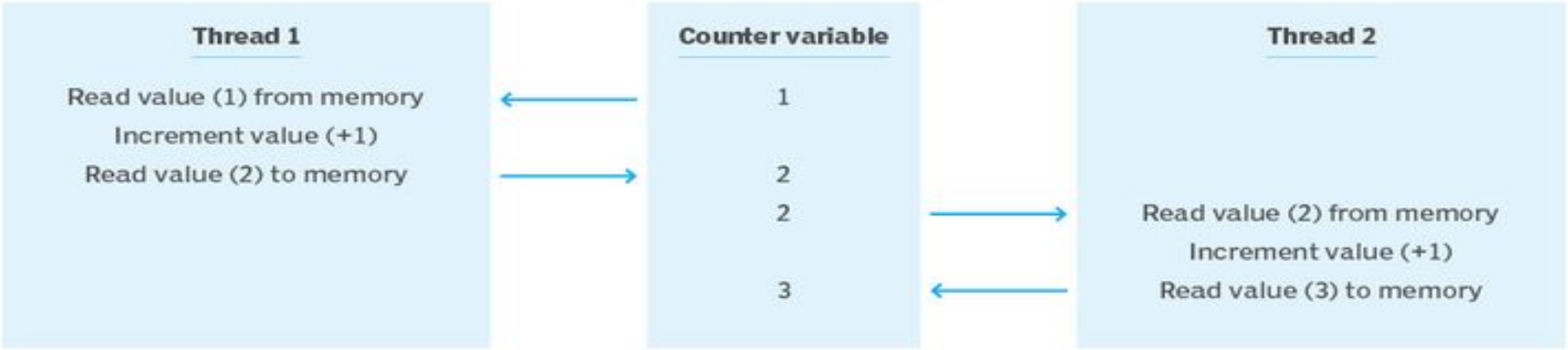
Contents

- Background
- The Critical-Section Problem
- Peterson's Solution
- Synchronization Hardware
- Semaphores
- Classic Problems of Synchronization

RV College of Engineering

Introduction

- Concurrent access to shared data may result in data inconsistency.
- Maintaining data consistency requires mechanisms to ensure the orderly execution of cooperating processes.
- Data access synchronization ensures that shared data do not lose consistency when they are updated by interacting processes-implemented by mutual exclusion



Introduction

- Control synchronization ensures that interacting processes perform their actions in a desired order.
- These two kinds of synchronization make up what we refer to as process synchronization.
- Atomic instructions support is provided to facilitate the synchronization.
- Term process is used to denote process and threads

The Critical-Section Problem

- One solution to process synchronization is the critical-section problem.
- Consider a system consisting of n processes {P0 , P1 , ..., Pn−1 }.
- Each process has a segment of code, called a critical section, in which the process may be changing common variables, updating a table, writing a file, and so on.
- When one process is executing in its critical section, no other process is allowed to execute in its critical section.
- no two processes are executing in their critical sections at the same time.

The Critical-Section Problem

- Each process must request permission to enter its critical section.
- The section of code implementing this request is the entry section.
- The critical section may be followed by an exit section. The remaining code is the remainder section.

```
do {
        entry section
            critical section
        exit section
            remainder section
} while (true);
```

The Critical-Section Problem

- A solution to the critical-section problem must satisfy the following three requirements:
- Mutual exclusion. If process Pi is executing in its critical section, then no other processes can be executing in their critical sections.
- Progress. If no process is executing in its critical section and some processes wish to enter their critical sections, then only those processes that are not executing in their remainder sections can participate in deciding which will enter its critical section next, and this selection cannot be postponed indefinitely.

The Critical-Section Problem

- Bounded waiting. There exists a bound, or limit, on the number of times that other processes are allowed to enter their critical sections after a process has made a request to enter its critical section and before that request is granted.

Peterson's Solution

- software-based solution to the critical-section problem known as Peterson's solution.
- Peterson's solution is restricted to two processes that alternate execution between their critical sections and remainder sections.

structure of process Pi in Peterson's solution

```
do {
    flag[i] = true;
    turn = j;
    while (flag[j] && turn == j);

        critical section

    flag[i] = false;

        remainder section

} while (true);
```

Synchronization Hardware

- techniques ranging from hardware to software-based APIs
- solutions are based on the premise of locking-protecting critical regions through the use of locks.
- Hardware solution- in a single-processor environment if we could prevent interrupts from occurring while a shared variable was being modified
- not as feasible in a multiprocessor environment.
- Solution: special hardware instructions that allow either to test and modify the content of a word or to swap the contents of two words atomically

Synchronization Hardware-test and set

- test and set() and compare and swap() instructions provides an abstraction of such type of instructions enabling atomicity

```
boolean test_and_set(boolean *target) {
   boolean rv = *target;
   *target = true;

   return rv;
}
```

```
do {
   while (test_and_set(&lock))
      ; /* do nothing */

      /* critical section */

   lock = false;

      /* remainder section */
} while (true);
```

Synchronization Hardware-compare and swap

```
int compare_and_swap(int *value, int expected, int new_value)
  int temp = *value;

  if (*value == expected)
     *value = new_value;

  return temp;
}
```

```
do {
  while (compare_and_swap(&lock, 0, 1) != 0)
     ; /* do nothing */

     /* critical section */

  lock = 0;

     /* remainder section */
} while (true);
```

Algorithm to solve issues of bounded waiting

```
do {
   waiting[i] = true;
   key = true;
   while (waiting[i] && key)
      key = test_and_set(&lock);
   waiting[i] = false;

      /* critical section */

   j = (i + 1) % n;
   while ((j != i) && !waiting[j])
      j = (j + 1) % n;

   if (j == i)
      lock = false;
   else
      waiting[j] = false;

      /* remainder section */
} while (true);
```

Semaphores

- A semaphore S is an integer variable that, apart from initialization, is accessed only through two standard atomic operations: wait() and signal().
- The wait() operation was originally termed P- to test
- signal() was originally called V- to increment

```
wait(S) {
    while (S <= 0)
        ; // busy wait
    S--;
}
```

```
signal(S) {
    S++;
}
```

Semaphores

- Two types of semaphores:
- Binary Semaphores: The value of a binary semaphore can range only between 0 and 1.
- Counting Semaphores : The value of a counting semaphore can range over an unrestricted domain, used to control access to a given resource consisting of a finite number of instances

Counting semaphores implementation

- The semaphore is initialized to the number of resources available.
- Each process that wishes to use a resource performs a wait() operation on the semaphore , which decrements the count.
- When a process releases a resource, it performs a signal() operation, which increments the count
- When the semaphore value is zero all the resources are occupied , after which process willing to use resources are blocked , it implements busy waiting

Semaphore implementation without busy waiting

- Definition of wait and signal suffers busy waiting
- To overcome that modify wait and signal as: When a process executes the wait() operation and finds that the semaphore value is not positive, it must wait. However, rather than engaging in busy waiting, the process can block itself.
- The block operation places a process into a waiting queue associated with the semaphore, and the state of the process is switched to the waiting state.
- Then control is transferred to the CPU scheduler, which selects another process to execute.

Semaphore implementation without busy waiting

- A process that is blocked, waiting on a semaphore S, should be restarted when some other process executes a signal() operation.
- The process is restarted by a wakeup() operation, which changes the process from the waiting state to the ready state. The process is then placed in the ready queue.

Semaphore structure, Wait, signal-definition

```
typedef struct {
    int value;
    struct process *list;
} semaphore;
```

```
signal(semaphore *S) {
    S->value++;
    if (S->value <= 0) {
        remove a process P from S->list;
        wakeup(P);
    }
}
```

```
wait(semaphore *S) {
    S->value--;
    if (S->value < 0) {
        add this process to S->list;
        block();
    }
}
```

Classic Problems of Synchronization

- The Bounded-Buffer Problem
- The Readers – Writers Problem
- The Dining-Philosophers Problem

Producer- Consumer Problem

- A producer process produces information that is consumed by a consumer process.
- a compiler may produce assembly code that is consumed by an assembler. assembler, in turn, may produce object modules that are consumed by the loader.
- Data processing applications: producers generate data-placed in shared data store- Consumers retrieve the data and perform processing.
- Solution to producer consumer is through bounded and unbounded buffer usage

The Bounded-Buffer Problem

```
do {
     . . .
   /* produce an item in next_produced */
     . . .
   wait(empty);
   wait(mutex);
     . . .
   /* add next_produced to the buffer */
     . . .
   signal(mutex);
   signal(full);
} while (true);
```

```
do {
   wait(full);
   wait(mutex);
     . . .
   /* remove an item from buffer to next_consumed */
     . . .
   signal(mutex);
   signal(empty);
     . . .
   /* consume the item in next_consumed */
     . . .
} while (true);
```

```
int n;
semaphore mutex = 1;
semaphore empty = n;
semaphore full = 0
```

The Readers – Writers Problem

- database is to be shared among several concurrent processes.
- Readers only read database, writers update the database
- Variations of reader writer problem:
- first readers–writers problem, requires that no reader be kept waiting unless a writer has already obtained permission to use the shared object.
- The second readers –writers problem requires that, once a writer is ready, that writer perform its write as soon as possible.

Solution to first reader writer problem

- Data structures used are :

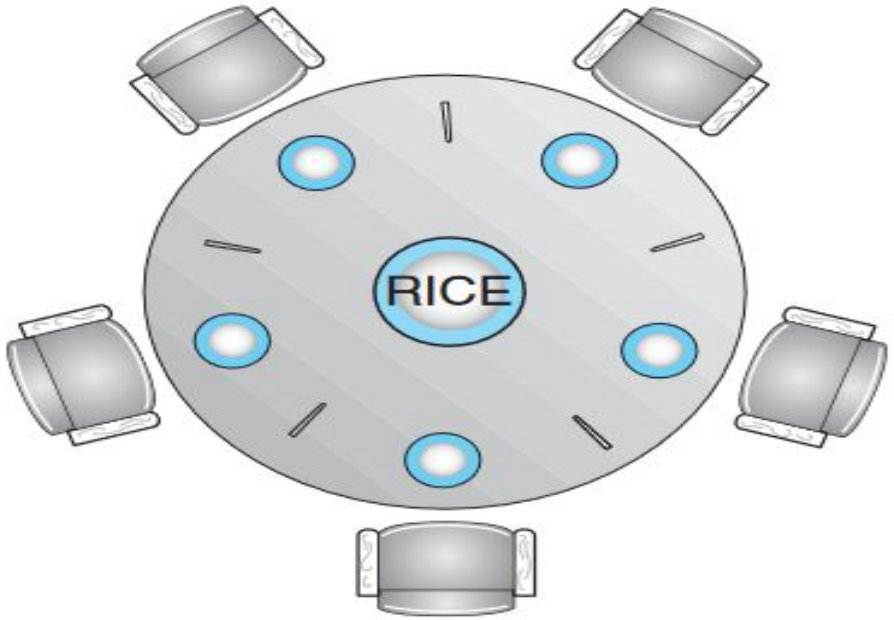
```
semaphore rw_mutex = 1;
semaphore mutex = 1;
int read_count = 0;
```

```
do {
   wait(rw_mutex);
      . . .
   /* writing is performed */
      . . .
   signal(rw_mutex);
} while (true);
```

```
do {
   wait(mutex);
   read_count++;
   if (read_count == 1)
      wait(rw_mutex);
   signal(mutex);
      . . .
   /* reading is performed */
      . . .
   wait(mutex);
   read_count--;
   if (read_count == 0)
      signal(rw_mutex);
   signal(mutex);
} while (true);
```

The Dining-Philosophers Problem

- Consider five philosophers who spend their lives thinking and eating.
- The philosophers share a circular table surrounded by five chairs, each belonging to one philosopher.



The Dining-Philosophers Problem

- When a philosopher thinks, he does not interact with colleagues.
- From time to time, a philosopher gets hungry and tries to pick up the two chopsticks that are closest to him (the chopsticks that are between him and his left and right neighbors).
- A philosopher may pick up only one chopstick at a time. He cannot pick up a chopstick that is already in the hand of a neighbor.
- When a hungry philosopher has both his chopsticks at the same time, he eats without releasing the chopsticks.
- When he is finished eating, he puts down both chopsticks and starts thinking again

The Dining-Philosophers Problem

- One simple solution is to represent each chopstick with a semaphore. A philosopher tries to grab a chopstick by executing a wait() operation on that semaphore.
- He releases his chopsticks by executing the signal() operation on the appropriate semaphores.
- Shared data structure is : semaphore chopstick[5];

The Dining-Philosophers Problem

```
do {
   wait(chopstick[i]);
   wait(chopstick[(i+1) % 5]);
      . . .
   /* eat for awhile */
      . . .
   signal(chopstick[i]);
   signal(chopstick[(i+1) % 5]);
      . . .
   /* think for awhile */
      . . .
} while (true);
```

The Dining-Philosophers Problem

- The algorithm can lead to deadlock
- Several possible remedies to the deadlock problem are replaced by:
- Allow at most four philosophers to be sitting simultaneously at the table.
- Allow a philosopher to pick up his chopsticks only if both chopsticks are available (to do this, he must pick them up in a critical section).
- Use an asymmetric solution—that is, an odd-numbered philosopher picks up first his left chopstick and then his right chopstick, whereas an even- numbered philosopher picks up his right chopstick and then his left chopstick.